B+ Tree Index Construction Strategies:

1. Introduction

This report details the implementation and testing of three distinct methods for constructing a B+ Tree index within the amlayer (Access Method Layer). Each method was designed to address a common operational scenario in a database system. The implementations were built upon the provided pflayer (Paged File Layer) and splayer (Slotted Page Layer) and were validated using custom test drivers.
The three primary objectives were:

1. **File Indexing:** Building an index from an existing, unsorted data file.
2. **Incremental Indexing:** Building an index concurrently as new records are inserted one by one into an empty file.
3. **Bulk-Loading:** Implementing a highly efficient, bottom-up construction method for a pre-sorted data file.

2. Implementation Details

2.1. Objective 1: Indexing from an Existing Unsorted File

Scenario: A database table (represented as a .tdb file) already contains a large volume of unsorted data. An administrator decides to add a new index to improve query performance.
Process:
The strategy is to perform a full scan of the data file and execute a standard, top-down B+ Tree insertion for every record encountered.
Implementation (testamcreateindex.c):

1. **Initialization:** A new, empty index file is created using AM_CreateIndex. This function prepares a single, empty leaf page at Page 0, which also serves as the initial root of the tree.
2. **Data File Scan:** The existing testdata_unsorted.tdb file is opened using PF_OpenFile.
3. **Page Iteration:** The test driver iterates through every page in the data file using PF_GetFirstPage and PF_GetNextPage.
4. **Record Iteration:** Within each data page, the driver iterates through all active record slots using the slotted-page function SP_GetNextRecord.
5. **Record ID Generation:** For each record found, a unique RecIdType (defined as an int) is generated by bit-packing the record's page number and slot ID into a single integer. This is the crucial formula for mapping a record's physical location to a storable recId: int recId = (pageNum << 16) | (slotID & 0xFFFF);

6. **Top-Down Insertion:** The record's key (extracted from the record data) and its newly generated recId are inserted into the index using the existing AM_InsertEntry function. This function performs a standard top-down traversal from the root, finds the correct leaf, inserts the key, and recursively handles any page splits that occur.

Analysis: This method successfully builds a valid B+ Tree. However, it is the least efficient, as each of the 10,000+ insertions triggers a separate top-down tree traversal from the root, leading to significant redundant I/O and page buffer requests.

2.2. Objective 2: Incremental Index Construction

Scenario: A new table is created, and data is inserted transactionally over time (e.g., by a live application). The index must be built and updated in lock-step with these data insertions.
Process:
The test driver simulates this behavior by creating both a new data file and a new index file simultaneously. It then loops, inserting one record into the data file, retrieving its new recId, and immediately inserting that recId into the index.
Implementation (testamcreateindexincremental.c):

1. **Initialization:** A new data file is created with PF_CreateFile and a new index file with AM_CreateIndex.
2. **Insertion Loop:** The test driver loops NUM_RECORDS times.
3. **Data Insertion:** Inside the loop, a new record is inserted into the data file.
    - The driver maintains a pointer to the "current" data page (sp_pageBuf).
    - SP_InsertRecord is called to add the new record to this page.
    - If SP_InsertRecord returns an error (e.g., SPE_PAGE_FULL), the current page is unfixed, a new page is allocated via PF_AllocPage, and the insertion is retried on this new page.
4. **Record ID Generation:** Once the record is successfully inserted, its (pageNum, slotID) are retrieved and packed into an int recId using the same bit-packing formula as in Objective 1.
5. **Index Insertion:** The new key and recId are immediately inserted into the index using AM_InsertEntry.

Analysis: This method correctly builds both the data and index files. It serves as a robust test of the B+ Tree's dynamic properties, validating that page splits, internal node creation, and increases in tree height are all handled correctly as the tree grows from an empty state.

2.3. Objective 3: Efficient Bulk-Loading from a Sorted File

Scenario: This is the most common case for initial data warehousing or populating a large table. A massive, pre-sorted dataset is available, and an index must be built as fast as possible.

Process:
This objective bypasses the slow, top-down AM_InsertEntry function entirely. A new function, AM_BulkLoad, was implemented to perform a highly efficient "bottom-up" construction. This algorithm builds the entire leaf level first, then recursively builds the internal node levels on top of it.

Implementation (am_bulkload.c):

Phase 1: Leaf Level Construction

1. **Root Reservation:** Page 0 is reserved for the final root. PF_AllocPage is called to allocate a new page (Page 1) to serve as the *first leaf page*. The global AM_LeftPageNum is set to 1.
2. **Data Scan:** The sorted testdata_sorted.tdb file is scanned record by record, page by page.
3. **Page Packing:** A helper function, PackLeafPage, is used to write key/recId pairs directly into the current leaf page buffer (e.g., Page 1). This is a simple memory copy that fills the page to 100% capacity, avoiding all B-Tree logic.
4. **Page Linking:** When a leaf page is full, it is written to disk. A *new* leaf page is allocated (e.g., Page 2). The previous leaf page's nextLeafPage pointer is then updated to point to this new page, creating the horizontal linked list at the leaf level.
5. **Key Promotion:** The **first key** that is inserted onto the *new* leaf page (Page 2) is "promoted" by saving it to a temporary parentEntries array. This array entry stores the key and the page number it points to (e.g., (key_from_page_2, page_2)).
6. This process repeats until all records are consumed. At the end, we have a complete, linked leaf level on disk and an in-memory array of all the keys that must be placed in the first internal level.

Phase 2: Internal Level Construction (Recursive)

1. **Recursive Function:** A recursive helper, BuildInternalLevel, is called. It takes the parentEntries list from the level below (e.g., the 31 entries from the leaf level) as its input.
2. **Internal Page Packing:** It allocates a new *internal page*. Its *first child pointer* is set to the leftmostChildPage (for the first call, this is Page 1).
3. **Entry Insertion:** It then packs the (key, ptr) pairs from the parentEntries list into this new internal page using another helper, PackInternalPage.
4. **Recursive Promotion:** If this internal page also fills up, a *new* internal page is allocated. The key that did not fit is promoted to the *next level up* by adding it to a *new* parentEntries list, which is then used for the next recursive call to BuildInternalLevel.
5. This recursion continues until the entire level fits on a *single internal page*. This single page is the new root.

Phase 3: Root Finalization

1. **The Bug:** An earlier implementation used Page 0 as the first leaf. This caused the final root (which was moved to Page 0) to have a child pointer pointing to itself (Page 0), resulting in an infinite loop and segmentation fault during AM_PrintTree.
2. **The Fix:** The final root page (e.g., Page 32) created by BuildInternalLevel is now correctly handled. Its contents are **copied** into Page 0. The temporary root page (Page

32) is then disposed of using PF_DisposePage.
3. This ensures that the final, canonical root of the B+ Tree is always at Page 0 and its child pointers correctly point down to the next level (e.g., Page 1, Page 31, etc.), not to itself.

Analysis: This method is by far the most efficient. It minimizes I/O by writing each page only once and results in a perfectly balanced, 100%-full tree. The test output confirmed that the leaf level was built, the internal levels were recursively created, the root was correctly placed at Page 0, and the AM_PrintTree function could successfully traverse the entire structure.

3. Conclusion

All three objectives were successfully implemented and tested. The project demonstrated the vast performance differences between transactional top-down insertions (AM_InsertEntry) and a set-based, bottom-up bulk-loading algorithm (AM_BulkLoad). The successful implementation of the bulk-loading function, including the critical fix for the root page self-reference, validates a complex and essential component of any practical access method layer.